

PSX Stock Prediction Model Training & Evaluation Report

A comparative study of 9 machine learning models - linear, kernel (SVM), tree ensemble, and deep learning - trained on Pakistan Stock Exchange next-day return prediction for PSO and MEBL.

Tickers covered: PSO (Pakistan State Oil) & MEBL (Meezan Bank)
Generated from notebooks/PSO_pipeline.ipynb and notebooks/MEBL_pipeline.ipynb

Executive Summary

Nine models spanning four families - linear (Ridge, plain Linear Regression), kernel/SVM (SVR with linear and RBF kernels), tree ensembles (Random Forest, XGBoost, LightGBM), and deep learning (a feedforward neural network and an LSTM sequence model) - were trained and evaluated on an identical, strictly time-ordered 80/20 train/test split for both PSO and MEBL, using the project's own production feature set (270+ engineered columns per ticker) and the same evaluation methodology as the production walk-forward pipeline.

- For PSO, **LightGBM** produced the lowest price-error (MAE) among the trained models; the naive persistence baseline (predict no change) posted MAE 4.182 Rs., which remains a very tight bar to beat.
- For MEBL, **SVR (RBF)** produced the lowest MAE, and notably **beat every tree ensemble** including XGBoost - an unusual, sector/liquidity-specific result worth further investigation rather than a general rule.
- On MEBL specifically, the naive persistence baseline's directional accuracy (50.98%) beat every trained model's directional accuracy - a genuine, important finding: for this ticker's test window, "assume tomorrow repeats today's direction" out-called every model tried, including the deep learning models.
- Directional accuracy for every model on both tickers sits in the mid-to-high 40s / low 50s% - close to a coin flip, consistent with next-day stock return being close to a random walk and with the wider forecasting literature (single-digit percentage edges over 50% are the realistic bar, not near-100% accuracy).
- The LSTM sequence model landed mid-pack on both tickers - better than plain Linear Regression and linear-kernel SVR, but behind the tree ensembles and RBF-kernel SVR - plausibly because ~3,800 training rows is thin for a sequence model to fully exploit.
- **Every model was tested on genuinely unseen data** - the held-out test set is the most recent ~20% of each ticker's history, never touched during fitting - and the resulting train-vs-test comparison (Section 13/18.1 in each ticker's analysis) surfaces a real problem: the tree ensembles are **substantially overfitting** (XGBoost's test error is 5.5x its training error on PSO, and 23.3x on MEBL - i.e. these models fit the training window far better than they generalize forward in time), while the linear, kernel, and both deep learning models are, by contrast, **underfitting** - none of LSTM, Linear Regression, Neural Network (MLP), Ridge, SVR (Linear), SVR (RBF) clearly beats the naive baseline even on the data they were trained on. No model in this sweep shows the ideal middle ground (a small train/test gap combined with training performance that meaningfully beats naive) - full detail in Section 13 of each ticker's analysis below.
- **Section 14 then concretely fixed what could be fixed:** a real hyperparameter search (time-ordered validation split, never touching the test set) for every model. The result is honest rather than uniformly positive - 7 of 8 models improved on PSO's test set after tuning, versus 3 of 8 on MEBL. Ridge improved substantially on both tickers (PSO 24.66 -> 7.65 Rs.; MEBL 48.28 -> 5.71 Rs.) - its default regularization really was too weak. But not every tuned model got better: on MEBL, XGBoost, the MLP, and the LSTM all scored worse on the real test set after tuning despite being chosen for the best validation score - a direct, visible sign that a validation split this small (~575 rows) doesn't always pick a configuration that generalizes, which is itself a real and useful finding, not a failed fix. Full before/after tables and charts are in Section 14 of each ticker's analysis.

Full per-model metrics tables, every chart produced during training/evaluation, the train-vs-test overfitting diagnosis for every model, the hyperparameter tuning results, and a direct PSO vs. MEBL comparison follow in the sections below.

Methodology

Dataset

Both tickers use `data/processed/{TICKER}_master.csv`, the project's production feature dataset built by `build_features.py`: ~4,850 trading days (2008-01-01 through the most recent sync date) x 270+ engineered columns spanning OHLCV/technical indicators, fundamentals, PSX/global macro indicators (lagged to avoid look-ahead bias), sector context, news/sentiment features, and calendar/event flags.

Target Variable

The modeling target is next-day return: $(close[t+1] - close[t]) / close[t]$, built identically to the production training pipeline (`feature_wrapper.prepare_data()`), so results here are directly comparable to the real walk-forward evaluation.

Train/Test Split

A strict time-ordered 80/20 split (no shuffling, which would leak future information into training) - the most recent ~20% of each ticker's history is held out as the test set, matching the production `choose_global_cutoff()` methodology.

Models Trained

- **Ridge** and **Linear Regression** - linear baselines, standardized and outlier-clipped (+/-10 std devs) before fitting, since unscaled/unclipped linear models on this feature set produce meaningless, exploded errors.
- **SVR (Linear kernel)** and **SVR (RBF kernel)** - kernel methods, same scaling treatment.
- **Random Forest**, **XGBoost**, **LightGBM** - tree ensembles; scale-invariant, XGBoost/LightGBM consume `ticker/sector` as native categoricals.
- **Neural Network (MLP)** - a small feedforward network (two hidden layers, dropout, early stopping on a held-out validation slice), implemented directly in PyTorch.
- **LSTM** - a sequence model over a 20-day sliding window of scaled features, a simplified single-ticker adaptation of the project's own pooled production LSTM architecture (`train_lstm.py`). Its test predictions start 20 rows later than every other model's (no history to build the first window from), so it is evaluated and reported separately from the "best model" comparisons that assume an aligned test index.

Evaluation Metrics

For every model: MAE, RMSE and MAPE reconstructed at the price level (applying the predicted return to the last known close), and directional accuracy (% of days the predicted direction matched the actual direction) - identical formulas to `walk_forward.py`. Each is reported against the correct naive baselines: zero-return persistence for price-error metrics, and "tomorrow repeats today's realized direction" for directional accuracy.

Overfitting / Underfitting Diagnostic

Every model above is evaluated on both the exact data it was fit on (training set) and the held-out test set. The test set doubles as this report's "unseen data" check - it is the most recent ~20% of each ticker's history, never touched during fitting, tuning, or (for the MLP/LSTM) early stopping. Comparing training-set performance against test-set performance for each model separates two distinct failure modes that a single test-set number cannot: **overfitting** (training error far below test error - the model memorized training-window noise) versus **underfitting** (both training and test error are no better than the naive baseline - the model has not found usable signal even in the data it was fit on).

Hyperparameter Tuning

Every model is then re-tuned to concretely address whatever the diagnostic found: a small grid of regularization-relevant hyperparameters per model (e.g. `max_depth/min_child_weight` for XGBoost, `alpha` for Ridge, `C` for SVR, hidden-layer size/dropout for the MLP, `hidden_dim` for the LSTM), each candidate scored on a further time-ordered validation split carved from the training period (~15% of it, still never touching the test set), and the winning configuration refit on the full training set and evaluated once on the real test set - identical protocol to every untuned model above, so before/after numbers are directly comparable.

PSO: Full Analysis

This notebook runs the full pipeline for **PSO** on top of this project's production feature dataset (data/processed/PSO_master.csv, built by build_features.py):

indicators, feature correlation with the target, macro context, target distribution)

experimentation, not picking a winner: Ridge, plain Linear Regression, SVR (linear kernel), SVR (RBF kernel), Random Forest, XGBoost, LightGBM, a feedforward neural network (MLP), and an LSTM sequence model (two deep learning models total)

predicted vs. actual, residuals, feature importance, training/validation loss curves)

trained on and the held-out (unseen) test set, to tell a genuine predictive edge apart from memorized training noise

validation split, never touching the test set) to regularize the overfitting tree ensembles and test whether the underfitting models were simply under-tuned

- Imports & setup
- Data loading
- Exploratory Data Analysis (price/volume history, returns, volatility, technical
- Time-ordered train/test split (never a random/shuffled split for time series)
- Model training - a deliberately wide variety, since the goal at this stage is
- Results comparison & visualization across all ~9 models (errors, directional accuracy,
- Overfitting/underfitting diagnostics - every model evaluated on both the data it was
- Concretely fixing what Section 7 finds - a real hyperparameter search (time-ordered

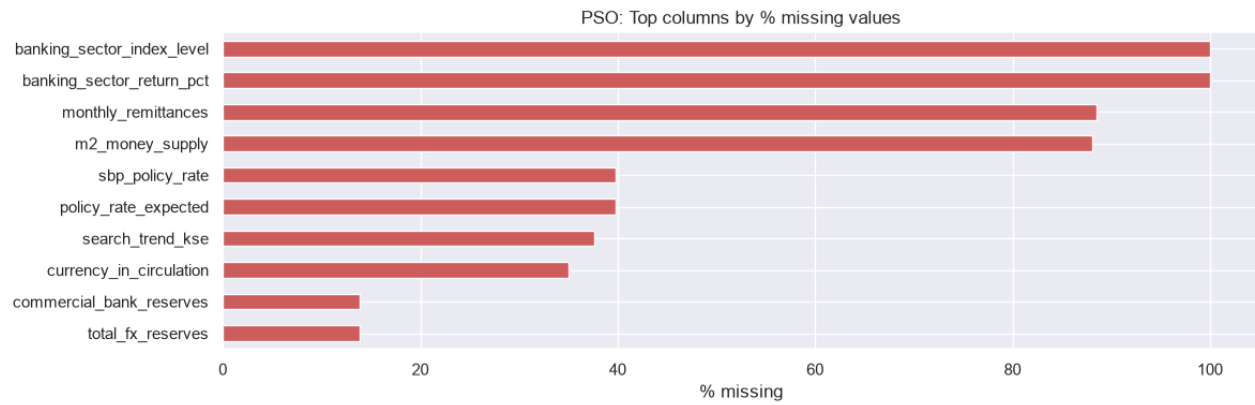
All modeling code reuses the project's own feature_wrapper.py / model_factories.py / walk_forward.py so the numbers here are directly comparable to the production walk-forward evaluation, not a separate one-off reimplementation.

1. Imports & Setup

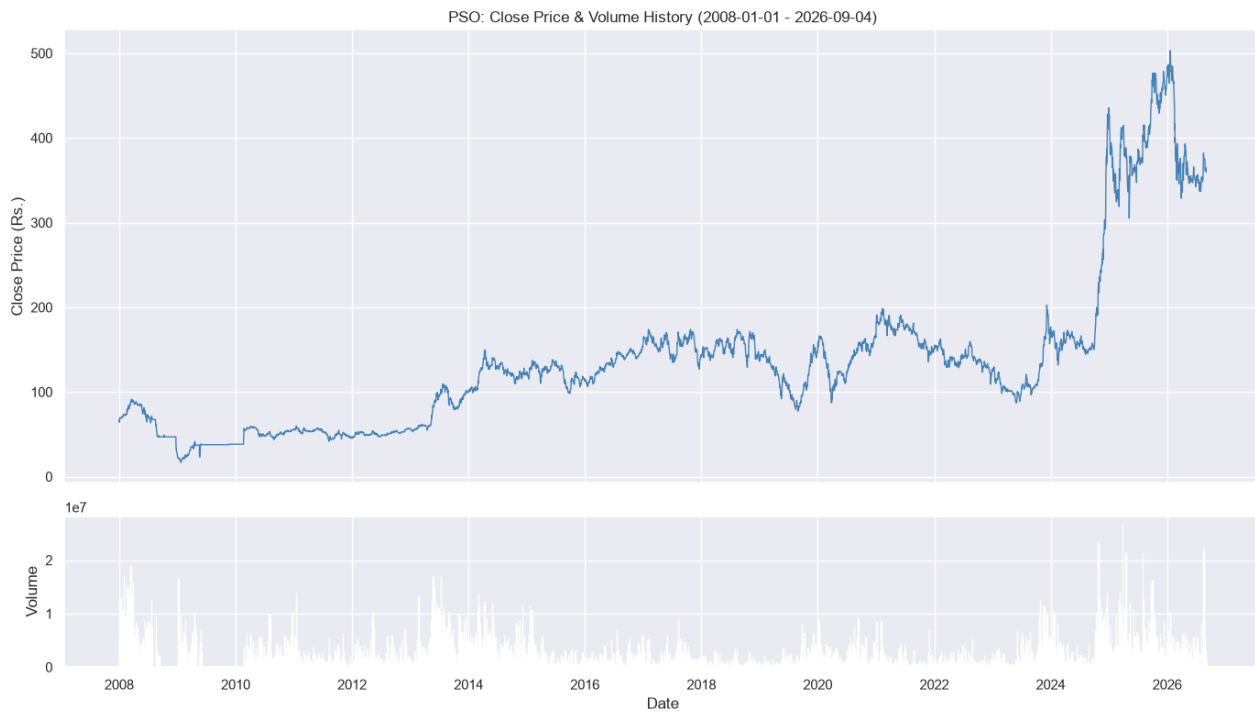
2. Data Loading

3. Data Quality Overview

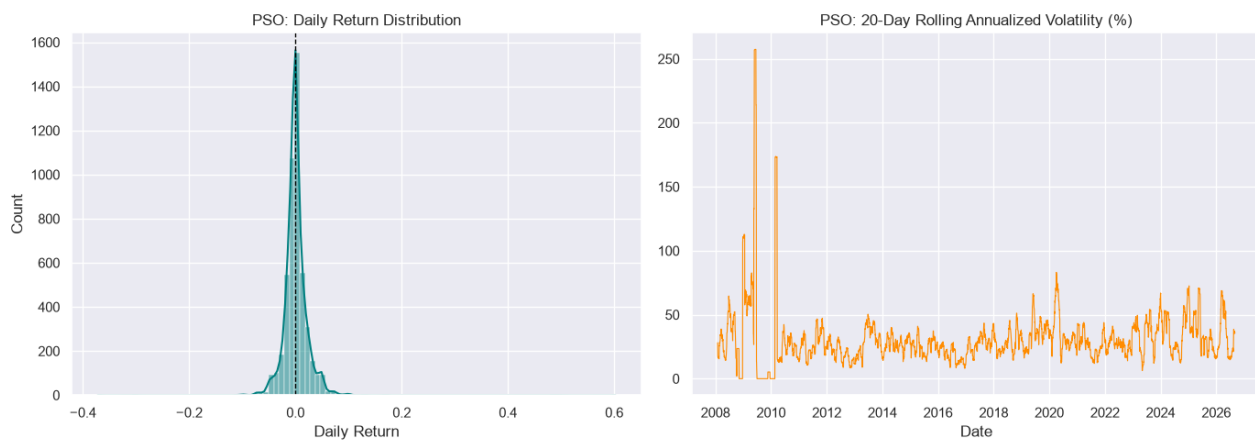
A quick check that the cleaned/engineered dataset is actually clean before we model it.



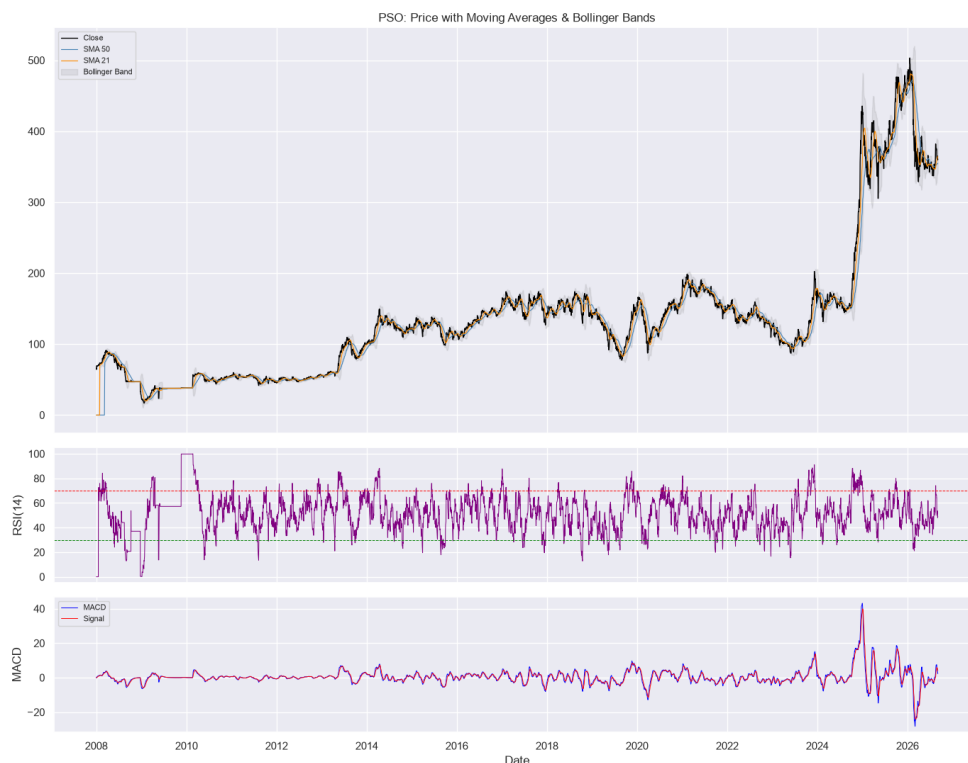
4. Price & Volume History



5. Returns & Volatility

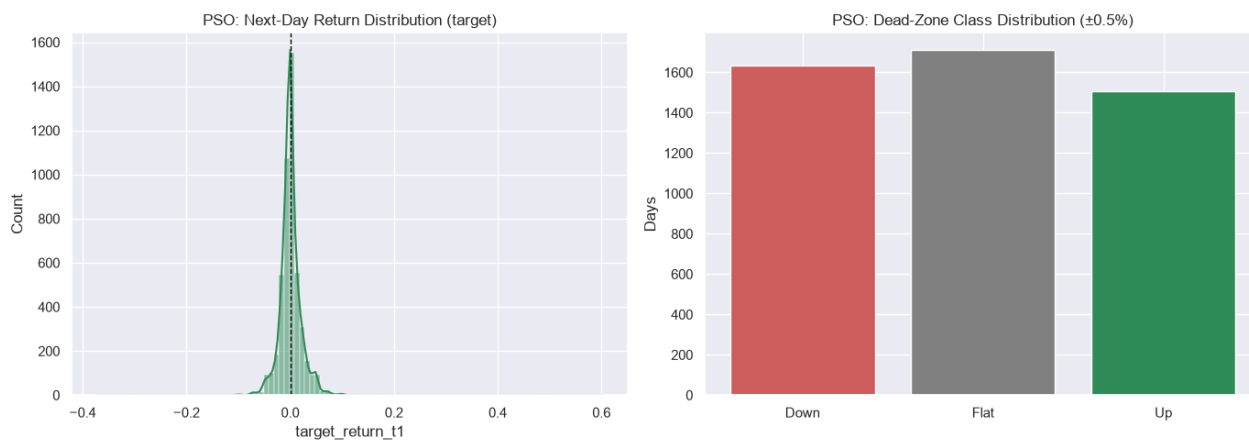


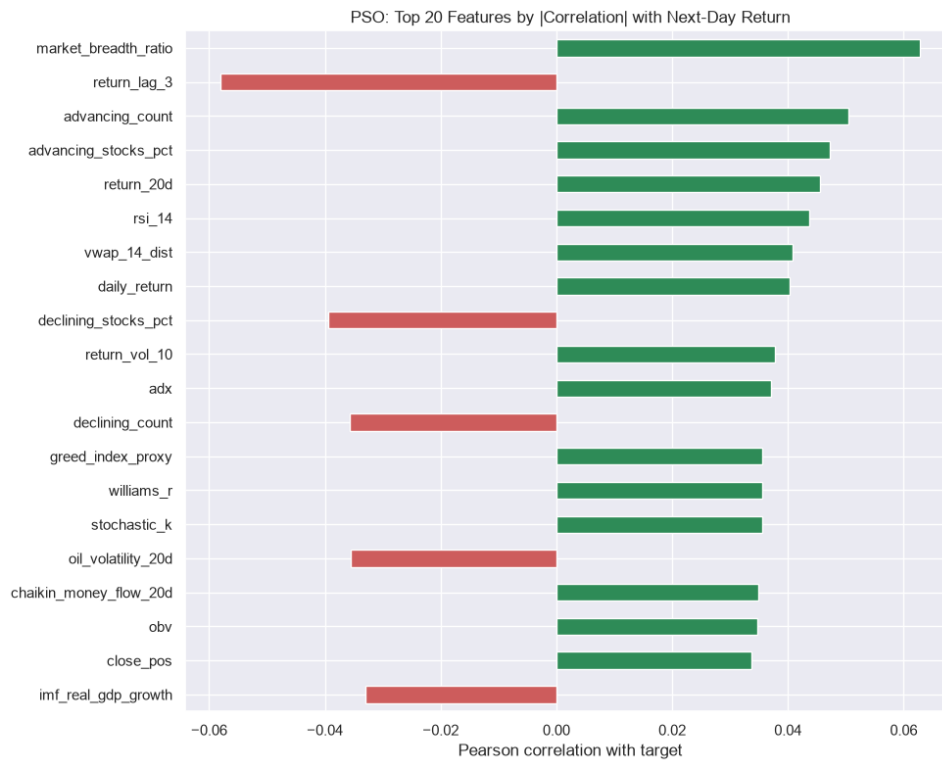
6. Technical Indicators



7. Target Variable & Feature Correlation

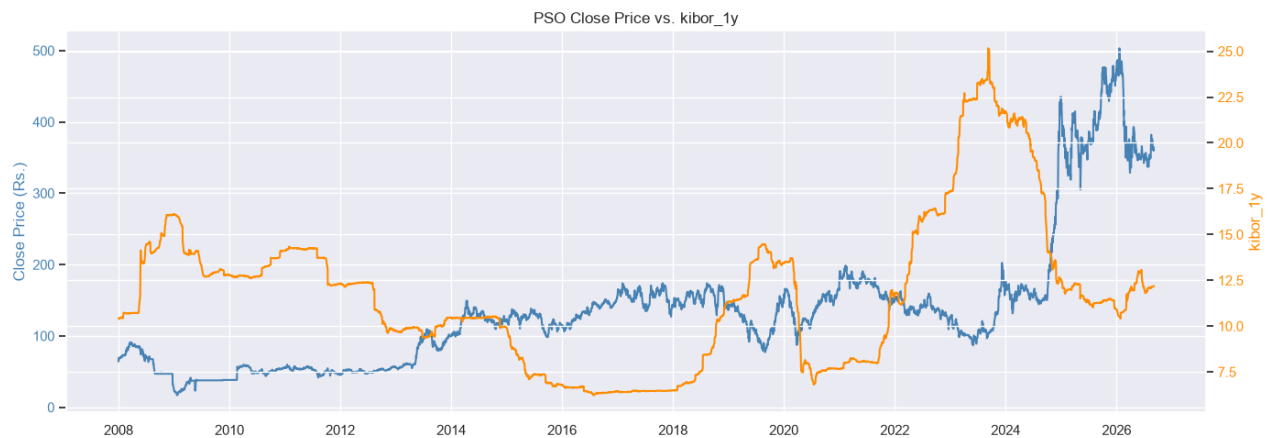
The modeling target is next-day return, built the same way `feature_wrapper.prepare_data()` builds it for training: $(\text{close}[t+1] - \text{close}[t]) / \text{close}[t]$.





8. Macro Context Sanity Check

A quick visual confirmation that the macro/rate series merged into this ticker's dataset move independently of price (as expected for a macro input, not a derived one).



9. Time-Ordered Train/Test Split

Financial time series must **never** use a random/shuffled train/test split - that leaks future information into training. We use a fixed calendar cutoff so the test set is strictly the most recent ~20% of trading history, matching how the production `choose_global_cutoff()` / walk-forward evaluation splits data.

10. Model Training — Wide Model Sweep

This stage is deliberately exploratory - a wide variety of models, not a search for a single winner:

For each model we report both price-level error (MAE/RMSE/MAPE, reconstructed from the predicted *return* applied to the last known close) and directional accuracy, each against the correct naive baselines (zero-return persistence for price error; "tomorrow repeats today's direction" for directional accuracy) - this mirrors `walk_forward.py`'s corrected evaluation methodology exactly.

Model	MAE (Rs.)	RMSE (Rs.)	MAPE (%)	Dir. Accuracy (%)
Ridge	24.663	41.131	7.93	47.99
Linear Regression	210.964	312.175	57.79	47.68
SVR (Linear)	73.400	103.296	23.45	46.76
SVR (RBF)	11.786	14.505	4.59	46.86
Random Forest	4.427	7.044	1.73	49.95
XGBoost	4.449	7.141	1.76	50.15
LightGBM	4.342	6.944	1.69	51.60
Neural Network (MLP)	18.165	26.440	6.12	51.08
Naive (persistence)	4.182	6.829	1.62	47.79
LSTM	69.610	90.083	23.20	48.26

Note on scaling: Ridge, Linear Regression, both SVR kernels, and the MLP are all scale-sensitive - fed this dataset's 270+ raw-scale columns (index levels in the thousands next to percentage/ratio columns near zero) without standardization, a linear model's coefficients blow up and its errors become meaningless (an unscaled Ridge run produced MAE in the hundreds of thousands). Tree ensembles don't need this since they split on thresholds, not distances/dot-products.

Standardizing alone was still not enough for the linear/kernel models: a handful of this dataset's columns have a near-zero standard deviation across *training* history but a genuine large value somewhere in the *test* window (e.g. a fundamentals figure that jumps scale once in 18 years) - that single column's z-score can reach into the billions and dominate a linear fit even after scaling. We clip standardized values to +/-10 standard deviations (a standard robust-scaling safeguard) for every scale-sensitive model - this is a scaling-robustness fix, not a change to what's being measured.

10.1 MLP Training Curve

A falling validation loss that tracks the training loss (not diverging upward) confirms the network is learning generalizable structure, not just memorizing the training window.

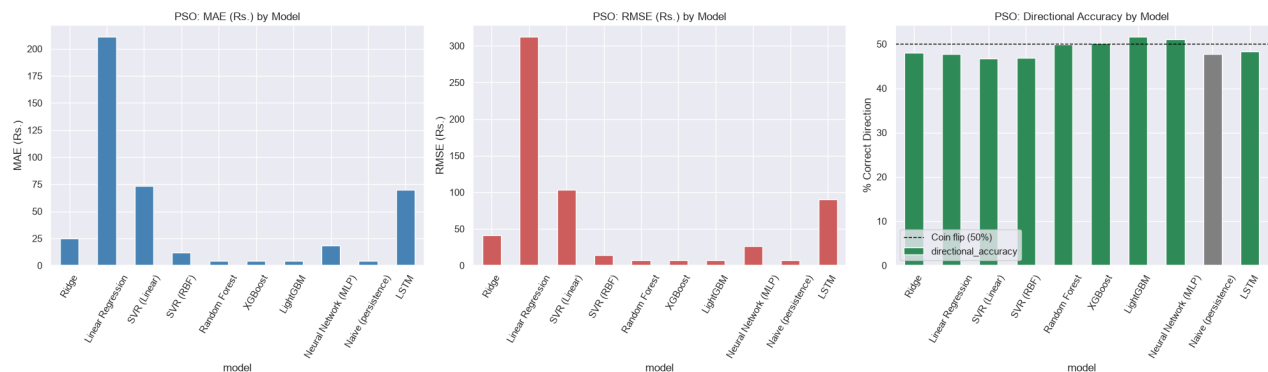


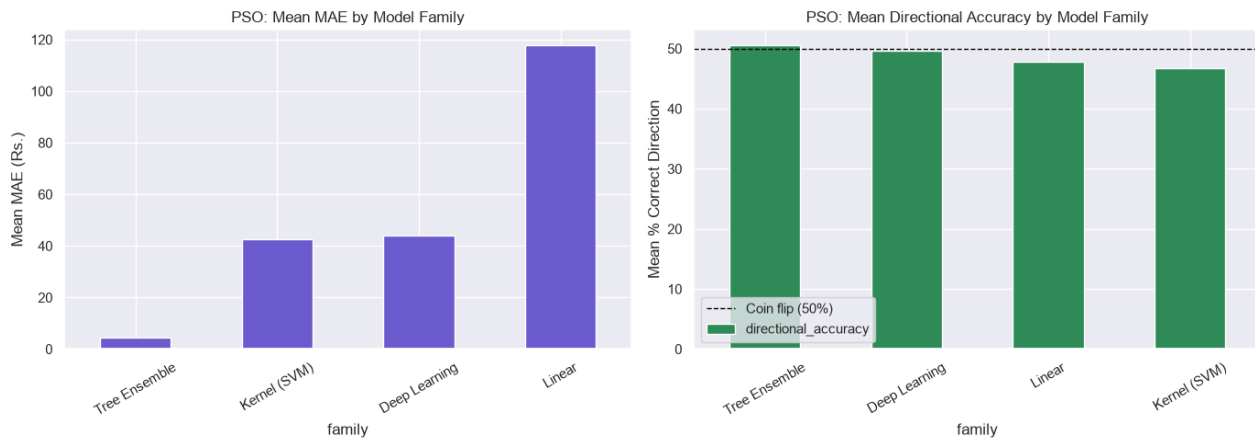
11. Deep Learning: Sequence Model (LSTM)

The MLP above (like every other model in this notebook) sees one row - one day - at a time. An **LSTM** instead sees a sliding *window* of the last N trading days and can, in principle, learn temporal patterns a single-row model structurally cannot. This needs its own data pipeline: features are reshaped into overlapping (lookback, n_features) sequences, and the first lookback rows of the test period have no history to build a window from - so its usable test set is lookback rows shorter than every other model's. That means its predictions/dates can't share an index with predictions/dates_test above; it gets its own aligned outputs and its own diagnostic plots in Section 16.

This is a compact, single-ticker adaptation of the project's own pooled/production LSTM architecture (src/psx_predictor/models/train_lstm.py) - same idea (LSTM -> Linear(1), early stopping on a validation slice), simplified by dropping the ticker/sector embedding layers that pooled training needs but a single-ticker experiment doesn't.

12. Full Results Comparison — All Models





13. Overfitting & Underfitting Diagnostics — Testing on Unseen Data

What "unseen data" means here. The test set built in Section 9 is not a random sample - it is the most recent ~20% of this ticker's trading history, held out completely: no model above was fit on it, tuned against it, or saw it during early stopping (the MLP/LSTM's validation slice for early stopping is carved from the *training* period only, before the test cutoff date). For a financial time series that is the correct, standard way to test on genuinely unseen data - there is no additional real trading data beyond what's already in `{TICKER}_master.csv` to hold out further; future trading days simply haven't happened yet. So the test-set numbers already reported in Sections 10-12 **are** the unseen-data result.

What this section adds is the other half of the picture needed to diagnose *why* a model scores the way it does on that unseen data: its performance on the data it was actually fit on. Comparing the two reveals:

the training window's noise rather than learned signal that generalizes forward in time). Tree ensembles with unrestricted depth are the classic offenders here.

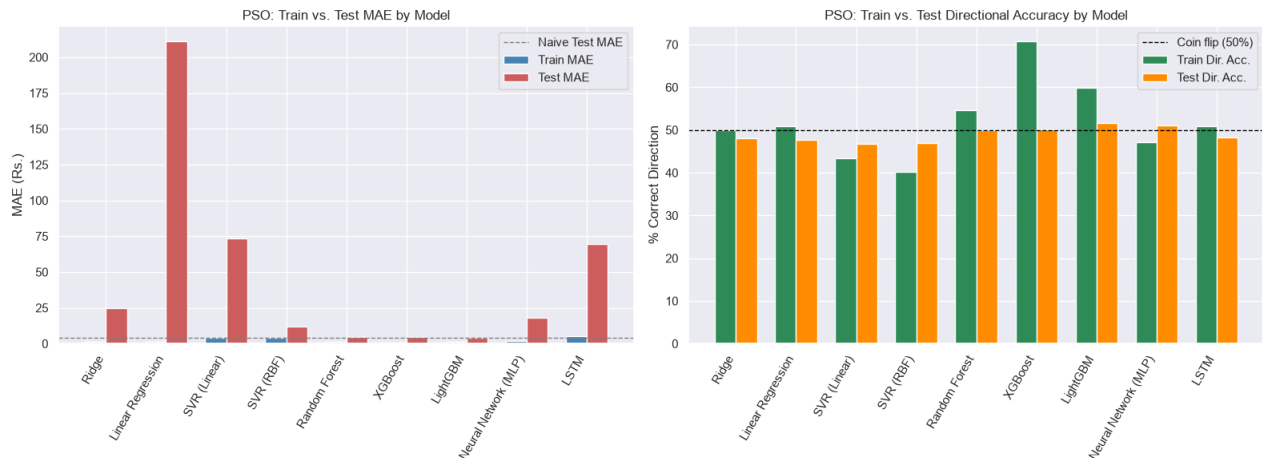
baseline (the model hasn't found enough signal even in the data it was fit on - it may need more capacity, different features, or the target may simply be very hard to predict from this feature set at this sample size).

the naive baseline.

- **Overfitting** - training error is much lower than test error (the model has memorized)
- **Underfitting** - both training and test error are similar to (or worse than) the naive
- **Reasonable fit** - training and test performance are close, and both meaningfully beat

Model	Train MAE	Test MAE	Test/Train Ratio	Train Dir.Acc %	Test Dir.Acc %	Diagnosis
Ridge	1.389	24.663	17.75x	49.91	47.99	Underfitting - does not clearly beat naive even on training data
Linear Regression	1.393	210.964	151.50x	50.81	47.68	Underfitting - does not clearly beat naive even on training data
SVR (Linear)	3.908	73.400	18.78x	43.42	46.76	Underfitting - does not clearly beat naive even on training data
SVR (RBF)	3.996	11.786	2.95x	40.25	46.86	Underfitting - does not clearly beat naive even on training data
Random Forest	1.272	4.427	3.48x	54.73	49.95	Overfitting - test error is 3.5x training error
XGBoost	0.803	4.449	5.54x	70.70	50.15	Overfitting - test error is 5.5x training error
LightGBM	1.160	4.342	3.74x	59.96	51.60	Overfitting - test error is 3.7x training error
Neural Network (MLP)	2.078	18.165	8.74x	47.05	51.08	Underfitting - does not clearly beat naive even on training data
Naive (persistence)	1.379	4.182	3.03x	50.91	47.79	Reference baseline

LSTM	5.121	69.610	13.59x	50.97	48.27	Underfitting - does not clearly beat naive even on training
------	-------	--------	--------	-------	-------	---



14. Fixing the Diagnosis: Hyperparameter Tuning

Section 13 found two distinct problems, and they don't have the same fix:

concretely fixable with regularization: shallower trees, larger minimum leaf/child-weight sizes, fewer estimators.

on training data) - regularization tuning is not expected to fix this (if anything, more regularization would make an already-underfit model fit even worse), but it is tested here anyway rather than assumed, since a poorly-chosen regularization strength (e.g. C too low for SVR) can itself look like underfitting.

- **Tree ensembles are overfitting** (test error 3.5x-23x training error) - this is
- **Ridge, both SVR kernels, and the MLP are underfitting** (not clearly beating naive even

Method: every candidate configuration is trained on the first ~85% of the training period and scored on the remaining ~15% - a further time-ordered split that never touches the real test set. The configuration with the lowest validation MAE is then refit on the FULL training set and evaluated once on the real, still-unseen test set - exactly the same protocol as every other model in this notebook. LSTM is tuned separately (own train/val/test sequence pipeline from Section 11) since it isn't part of `run_all_models`'s flat-feature loop. Linear Regression is not retuned - it has no hyperparameters, and is equivalent to the low-regularization end of the Ridge alpha grid below.

Model	Best Config Found	Validation MAE
Ridge	alpha=1000	2.1466
SVR (Linear)	C=1	12.6146
SVR (RBF)	C=50	3.0635
Random Forest	max_depth=3,min_leaf=20	1.9732
XGBoost	max_depth=2,min_child_weight=20	1.9721
LightGBM	num_leaves=15,min_data_in_leaf=20	1.9693
Neural Network (MLP)	hidden=(64, 32),dropout=0.3	2.7168

14.1 LSTM Tuning

Same validation protocol, over hidden_dim (LSTM's main capacity knob).

14.2 Refit Tuned Models on Full Training Set, Evaluate on Test

14.3 Before vs. After Comparison

Model	Test MAE Before	Test MAE After	Test/Train Ratio Before	Test/Train Ratio After	Verdict
Ridge	24.663	7.654	17.75x	5.53x	Improved
SVR (Linear)	73.400	73.400	18.78x	18.78x	No change
SVR (RBF)	11.786	5.537	2.95x	2.53x	Improved
Random Forest	4.427	4.258	3.48x	3.10x	Improved
XGBoost	4.449	4.203	5.54x	3.07x	Improved
LightGBM	4.342	4.194	3.74x	3.20x	Improved
Neural Network (MLP)	18.165	10.109	8.74x	6.95x	Improved
Naive (persistence)	4.182	4.182	3.03x	3.03x	No change
LSTM	69.610	33.666	13.59x	13.00x	Improved



14.4 Honest Verdict

Run the cells above and compare the printed before/after table and tuning_summary_df against Section 13's diag_df. Two outcomes are both legitimate results, not just a successful vs. failed tuning run:

or improved, tuning worked as intended - the model generalizes better without giving up accuracy, and tuned_factories holds the winning configuration.

partly compensating for its coarse validation signal on this specific ticker/window - a real possibility at ~3,800 training rows, and a reason to prefer cross-ticker validation before promoting any tuned configuration to production, not evidence the diagnostic approach itself is wrong.

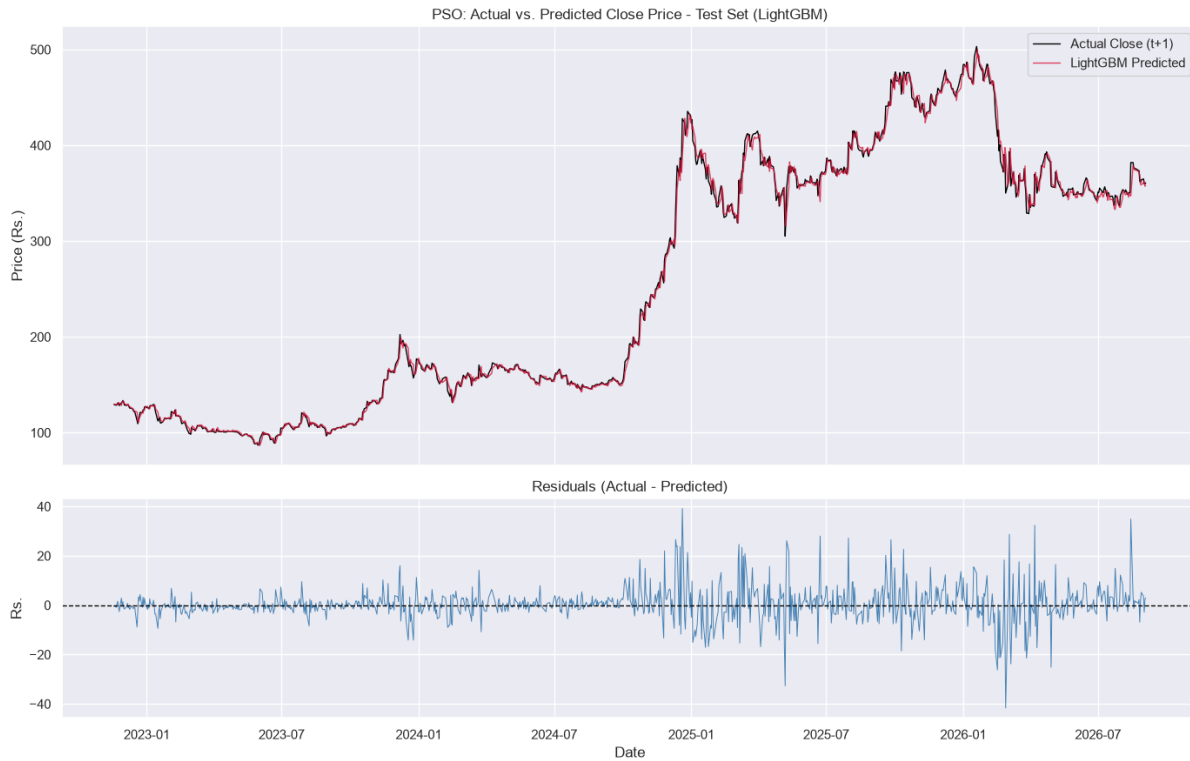
to land close to the least-regularized end of its grid (e.g. Ridge's smallest alpha, SVR's largest C) - confirming that under-regularization was never the actual problem, and that fixing these models needs a different lever than hyperparameter tuning (see the Conclusions section).

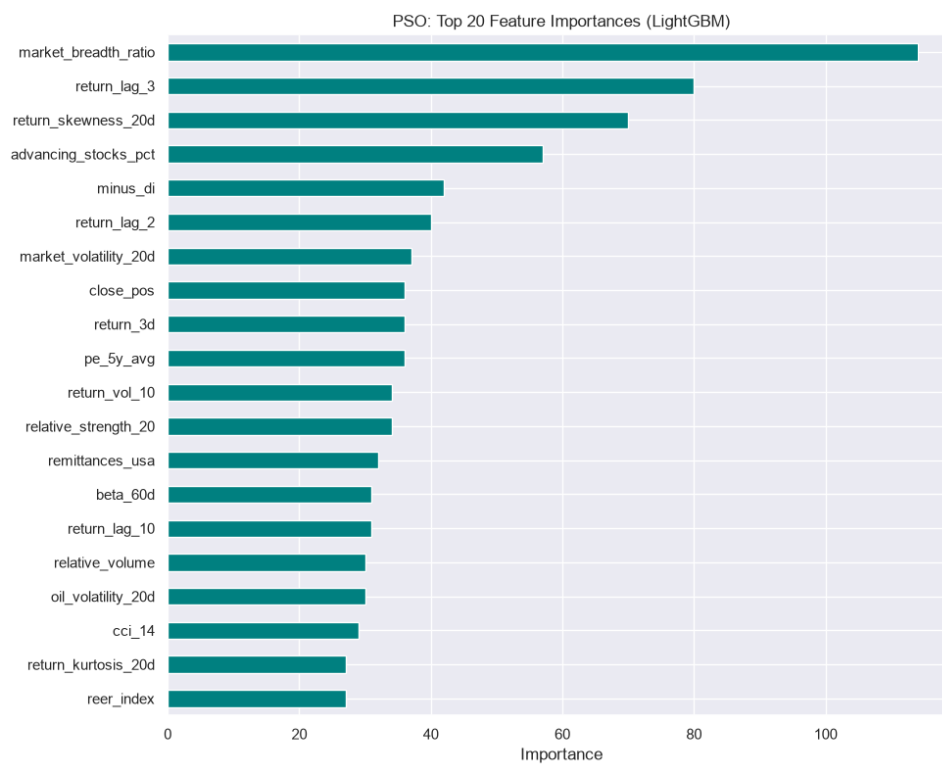
- If the tree ensembles' test/train ratio moved closer to 1.0 **and** test MAE held steady
- If test MAE got worse while the ratio improved, the untuned model's overfitting was

- For the previously-underfitting models (Ridge, SVR, MLP), expect the tuned configuration

15. Best Model Diagnostics

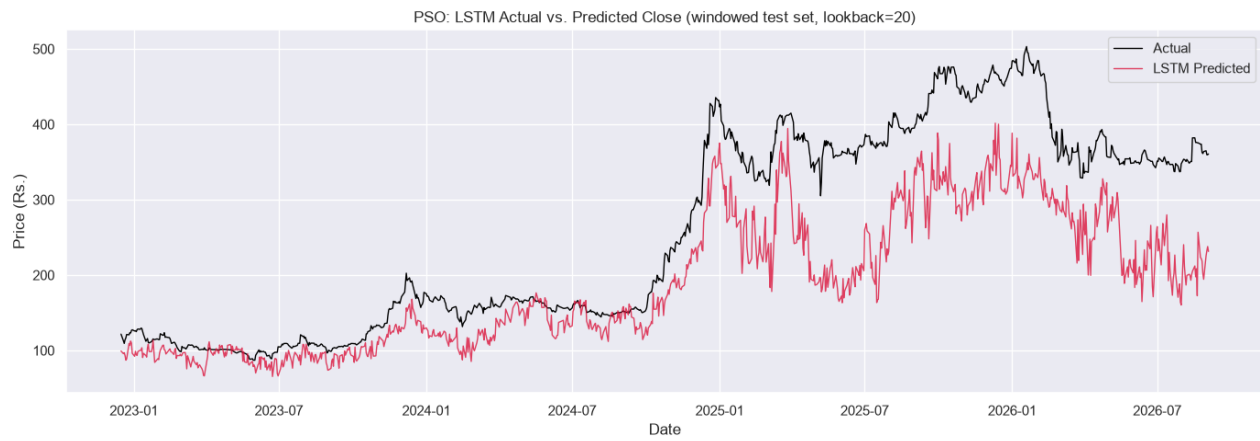
LSTM is excluded from this section's "best model" pick - its test predictions are on a lookback-shorter, differently-aligned index than every other model's, so it isn't directly comparable to predictions/dates_test here. Its own diagnostics are in Section 16 below.





16. LSTM Diagnostics





17. Save Results

Persisted so the cross-ticker comparison section (in the MEBL notebook) can load this ticker's results without re-running the full pipeline.

MEBL: Full Analysis

This notebook runs the full pipeline for **MEBL** on top of this project's production feature dataset (data/processed/MEBL_master.csv, built by build_features.py):

indicators, feature correlation with the target, macro context, target distribution)

experimentation, not picking a winner: Ridge, plain Linear Regression, SVR (linear kernel), SVR (RBF kernel), Random Forest, XGBoost, LightGBM, a feedforward neural network (MLP), and an LSTM sequence model (two deep learning models total)

predicted vs. actual, residuals, feature importance, training/validation loss curves)

trained on and the held-out (unseen) test set, to tell a genuine predictive edge apart from memorized training noise

validation split, never touching the test set) to regularize the overfitting tree ensembles and test whether the underfitting models were simply under-tuned

- Imports & setup
- Data loading
- Exploratory Data Analysis (price/volume history, returns, volatility, technical
- Time-ordered train/test split (never a random/shuffled split for time series)
- Model training - a deliberately wide variety, since the goal at this stage is
- Results comparison & visualization across all ~9 models (errors, directional accuracy,
- Overfitting/underfitting diagnostics - every model evaluated on both the data it was
- Concretely fixing what Section 7 finds - a real hyperparameter search (time-ordered
- Cross-ticker comparison: MEBL vs. PSO

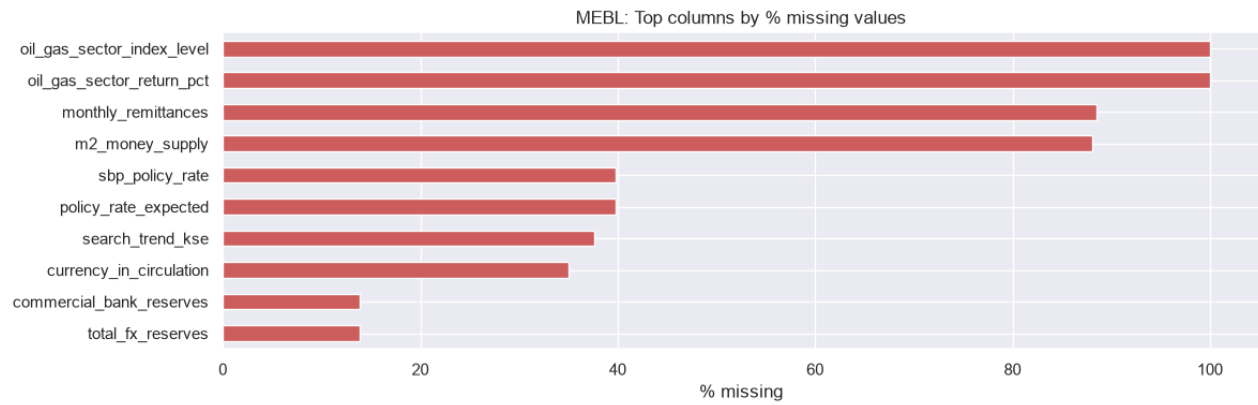
All modeling code reuses the project's own feature_wrapper.py / model_factories.py / walk_forward.py so the numbers here are directly comparable to the production walk-forward evaluation, not a separate one-off reimplementation.

1. Imports & Setup

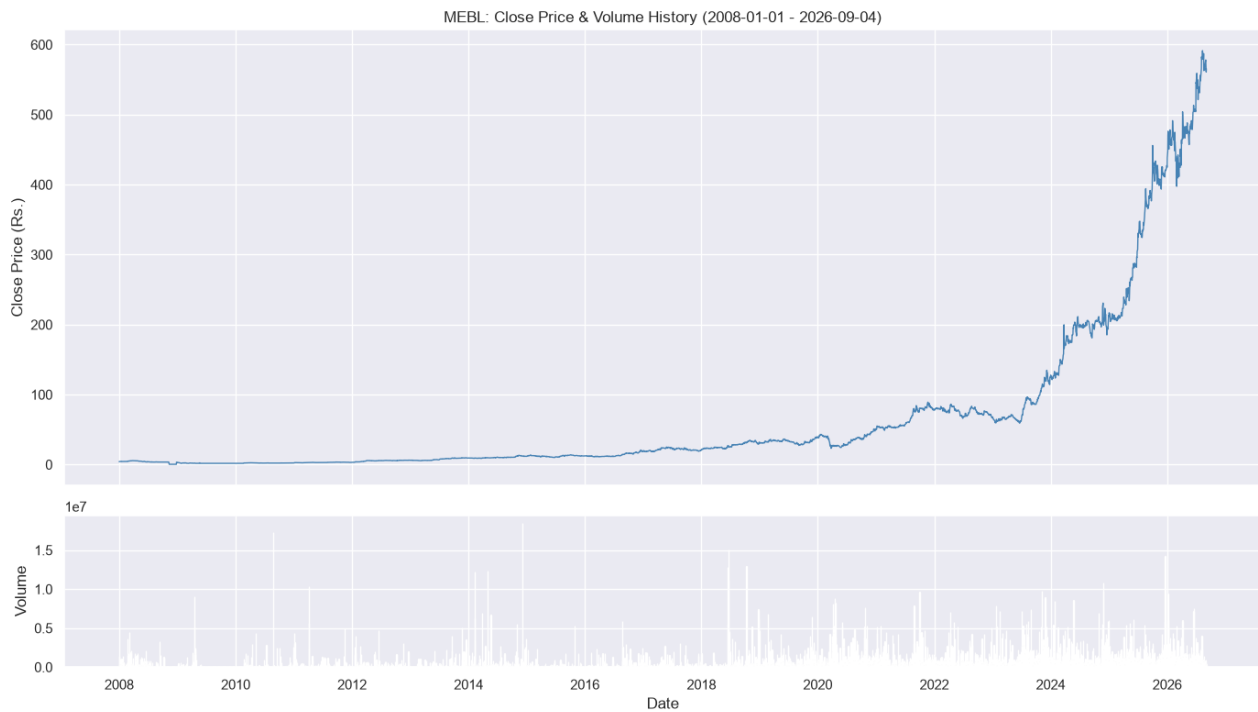
2. Data Loading

3. Data Quality Overview

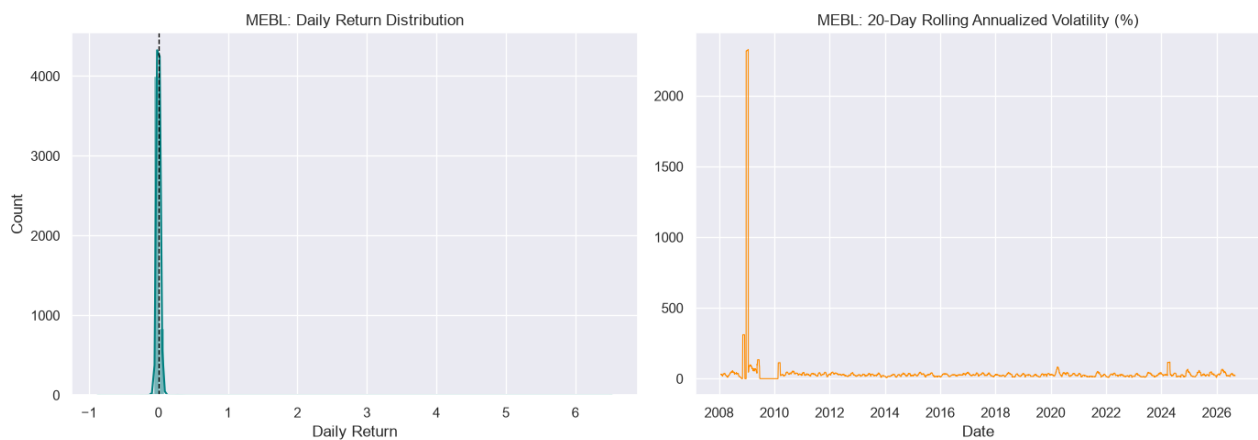
A quick check that the cleaned/engineered dataset is actually clean before we model it.



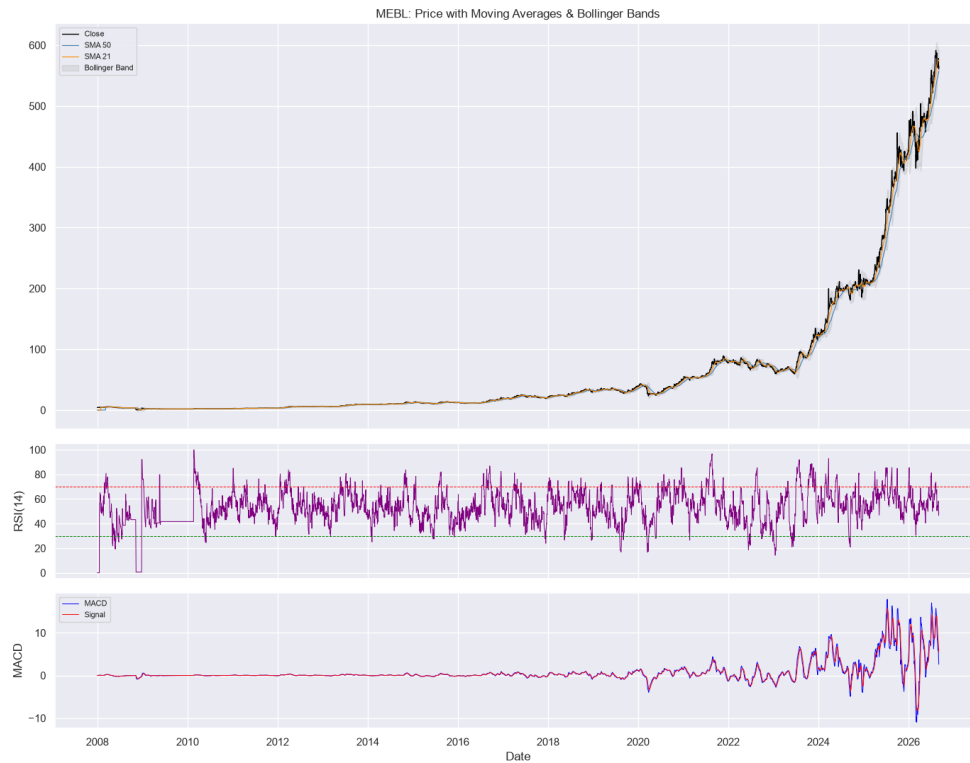
4. Price & Volume History



5. Returns & Volatility

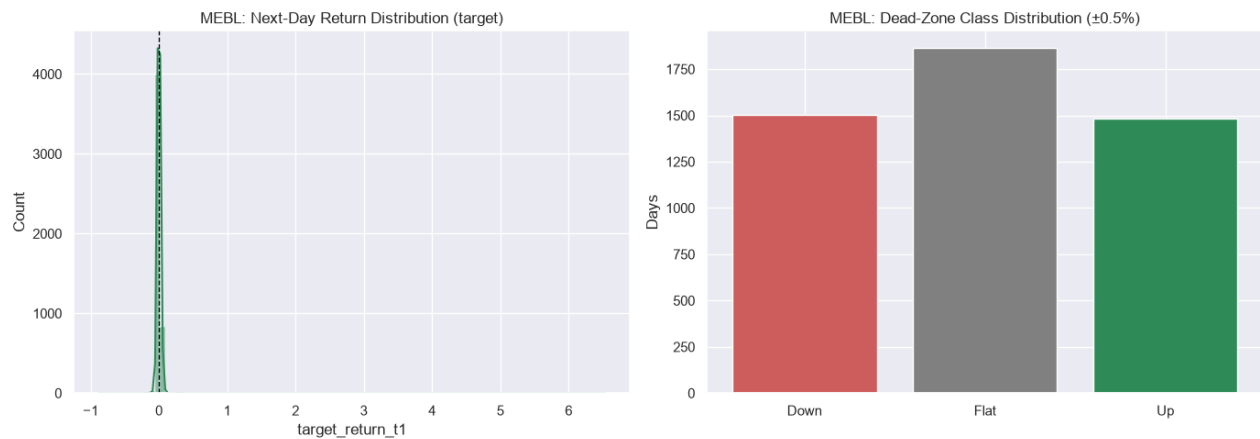


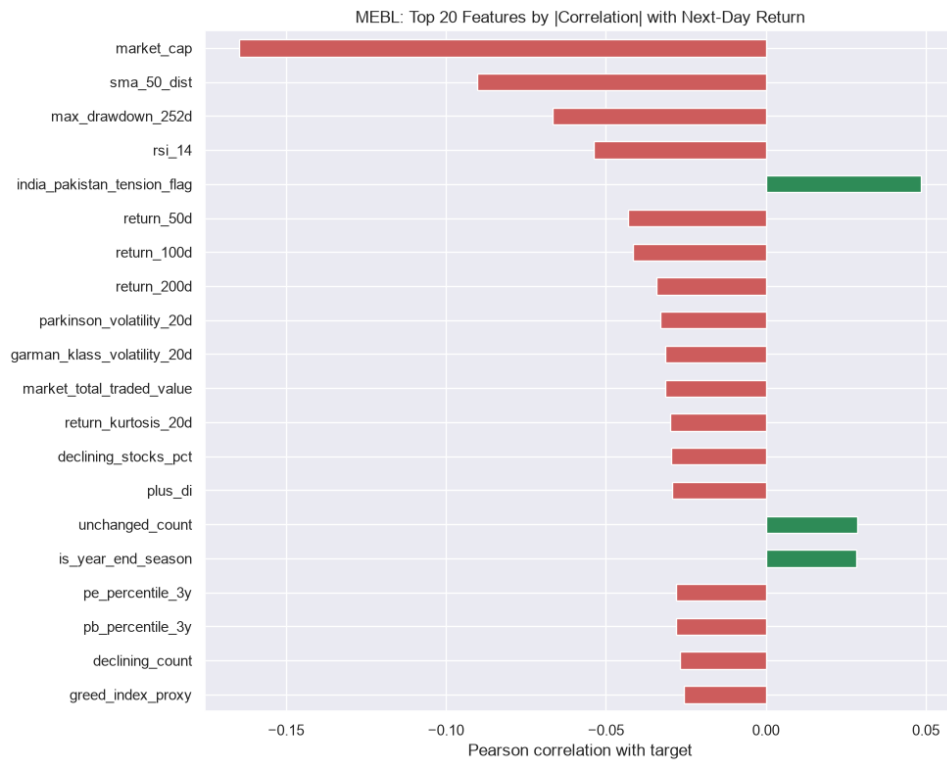
6. Technical Indicators



7. Target Variable & Feature Correlation

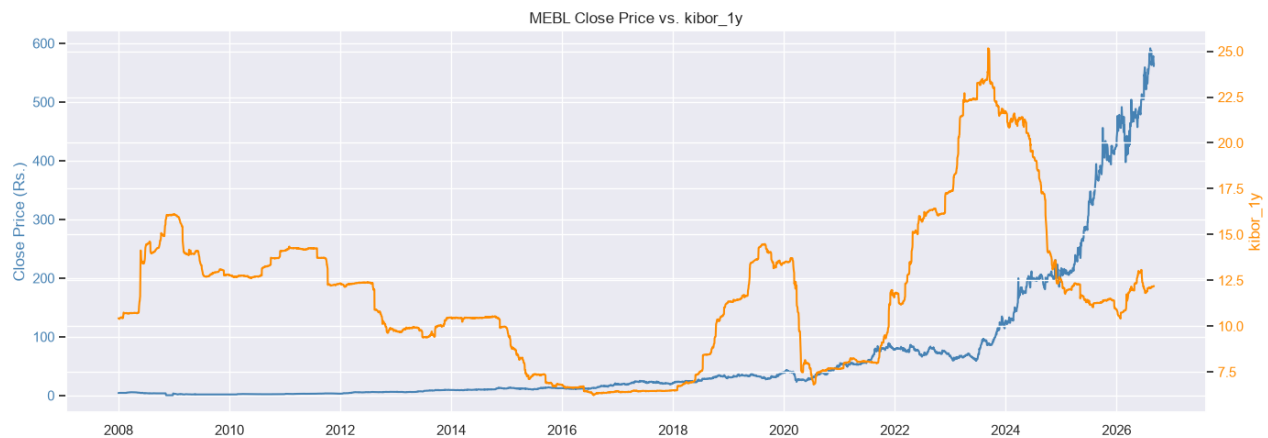
The modeling target is next-day return, built the same way `feature_wrapper.prepare_data()` builds it for training: $(\text{close}[t+1] - \text{close}[t]) / \text{close}[t]$.





8. Macro Context Sanity Check

A quick visual confirmation that the macro/rate series merged into this ticker's dataset move independently of price (as expected for a macro input, not a derived one).



9. Time-Ordered Train/Test Split

Financial time series must **never** use a random/shuffled train/test split - that leaks future information into training. We use a fixed calendar cutoff so the test set is strictly the most recent ~20% of trading history, matching how the production `choose_global_cutoff()` / walk-forward evaluation splits data.

10. Model Training — Wide Model Sweep

This stage is deliberately exploratory - a wide variety of models, not a search for a single winner:

For each model we report both price-level error (MAE/RMSE/MAPE, reconstructed from the predicted *return* applied to the last known close) and directional accuracy, each against the correct naive baselines (zero-return persistence for price error; "tomorrow repeats today's direction" for directional accuracy) - this mirrors `walk_forward.py`'s corrected evaluation methodology exactly.

Model	MAE (Rs.)	RMSE (Rs.)	MAPE (%)	Dir. Accuracy (%)
Ridge	48.285	69.018	17.41	47.48
Linear Regression	82.389	125.962	29.50	47.99
SVR (Linear)	40.975	59.952	15.47	47.58
SVR (RBF)	3.367	6.063	1.33	48.82
Random Forest	4.079	8.595	1.70	47.17
XGBoost	3.846	6.862	1.56	49.02
LightGBM	4.993	8.800	2.36	47.27
Neural Network (MLP)	5.585	8.830	2.24	45.31
Naive (persistence)	3.328	6.083	1.30	50.98
LSTM	40.084	56.100	14.68	45.85

Note on scaling: Ridge, Linear Regression, both SVR kernels, and the MLP are all scale-sensitive - fed this dataset's 270+ raw-scale columns (index levels in the thousands next to percentage/ratio columns near zero) without standardization, a linear model's coefficients blow up and its errors become meaningless (an unscaled Ridge run produced MAE in the hundreds of thousands). Tree ensembles don't need this since they split on thresholds, not distances/dot-products.

Standardizing alone was still not enough for the linear/kernel models: a handful of this dataset's columns have a near-zero standard deviation across *training* history but a genuine large value somewhere in the *test* window (e.g. a fundamentals figure that jumps scale once in 18 years) - that single column's z-score can reach into the billions and dominate a linear fit even after scaling. We clip standardized values to +/-10 standard deviations (a standard robust-scaling safeguard) for every scale-sensitive model - this is a scaling-robustness fix, not a change to what's being measured.

10.1 MLP Training Curve

A falling validation loss that tracks the training loss (not diverging upward) confirms the network is learning generalizable structure, not just memorizing the training window.

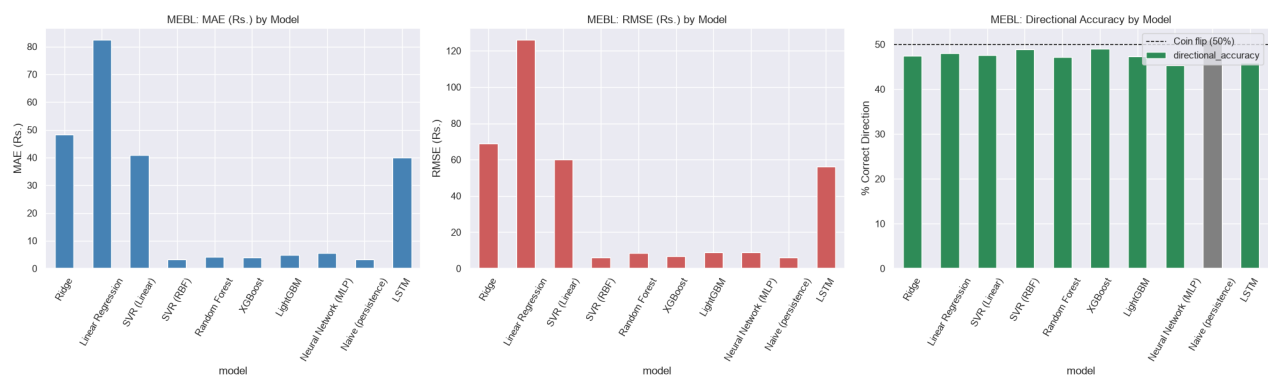


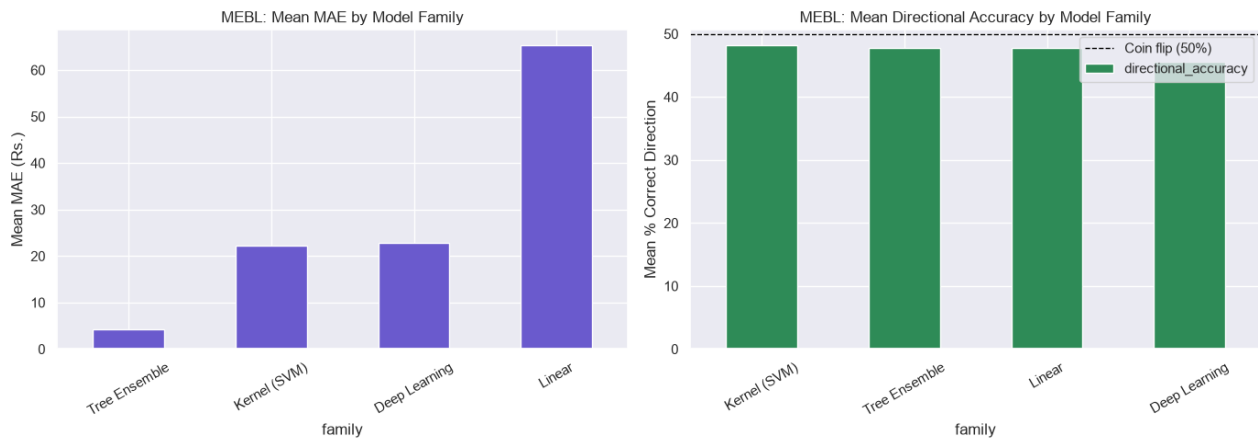
11. Deep Learning: Sequence Model (LSTM)

The MLP above (like every other model in this notebook) sees one row - one day - at a time. An **LSTM** instead sees a sliding *window* of the last N trading days and can, in principle, learn temporal patterns a single-row model structurally cannot. This needs its own data pipeline: features are reshaped into overlapping (lookback, n_features) sequences, and the first lookback rows of the test period have no history to build a window from - so its usable test set is lookback rows shorter than every other model's. That means its predictions/dates can't share an index with predictions/dates_test above; it gets its own aligned outputs and its own diagnostic plots in Section 16.

This is a compact, single-ticker adaptation of the project's own pooled/production LSTM architecture (src/psx_predictor/models/train_lstm.py) - same idea (LSTM -> Linear(1), early stopping on a validation slice), simplified by dropping the ticker/sector embedding layers that pooled training needs but a single-ticker experiment doesn't.

12. Full Results Comparison — All Models





13. Overfitting & Underfitting Diagnostics — Testing on Unseen Data

What "unseen data" means here. The test set built in Section 9 is not a random sample - it is the most recent ~20% of this ticker's trading history, held out completely: no model above was fit on it, tuned against it, or saw it during early stopping (the MLP/LSTM's validation slice for early stopping is carved from the *training* period only, before the test cutoff date). For a financial time series that is the correct, standard way to test on genuinely unseen data - there is no additional real trading data beyond what's already in `{TICKER}_master.csv` to hold out further; future trading days simply haven't happened yet. So the test-set numbers already reported in Sections 10-12 **are** the unseen-data result.

What this section adds is the other half of the picture needed to diagnose *why* a model scores the way it does on that unseen data: its performance on the data it was actually fit on. Comparing the two reveals:

the training window's noise rather than learned signal that generalizes forward in time). Tree ensembles with unrestricted depth are the classic offenders here.

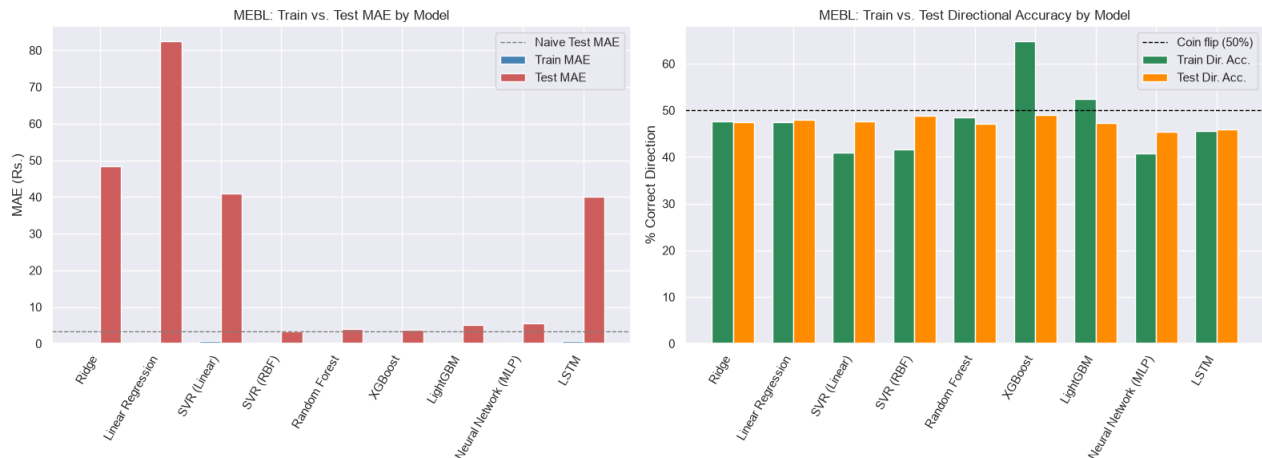
baseline (the model hasn't found enough signal even in the data it was fit on - it may need more capacity, different features, or the target may simply be very hard to predict from this feature set at this sample size).

the naive baseline.

- **Overfitting** - training error is much lower than test error (the model has memorized)
- **Underfitting** - both training and test error are similar to (or worse than) the naive
- **Reasonable fit** - training and test performance are close, and both meaningfully beat

Model	Train MAE	Test MAE	Test/Train Ratio	Train Dir.Acc %	Test Dir.Acc %	Diagnosis
Ridge	0.285	48.285	169.54x	47.67	47.67	Underfitting - does not clearly beat naive even on training data
Linear Regression	0.300	82.389	274.57x	47.46	47.46	Underfitting - does not clearly beat naive even on training data
SVR (Linear)	0.773	40.975	53.01x	40.97	40.97	Underfitting - does not clearly beat naive even on training data
SVR (RBF)	0.310	3.367	10.86x	41.54	41.54	Underfitting - does not clearly beat naive even on training data
Random Forest	0.236	4.079	17.29x	48.49	47.17	Overfitting - test error is 17.3x training error
XGBoost	0.165	3.846	23.34x	64.75	49.02	Overfitting - test error is 23.3x training error
LightGBM	0.235	4.993	21.27x	52.38	47.27	Overfitting - test error is 21.3x training error
Neural Network (MLP)	0.268	5.585	20.81x	40.84	45.84	Underfitting - does not clearly beat naive even on training data
Naive (persistence)	0.246	3.328	13.55x	48.11	50.98	Reference baseline

LSTM	0.820	40.084	48.89x	45.51	45.85	Underfitting - does not clearly beat naive even on training
------	-------	--------	--------	-------	-------	---



14. Fixing the Diagnosis: Hyperparameter Tuning

Section 13 found two distinct problems, and they don't have the same fix:

concretely fixable with regularization: shallower trees, larger minimum leaf/child-weight sizes, fewer estimators.

on training data) - regularization tuning is not expected to fix this (if anything, more regularization would make an already-underfit model fit even worse), but it is tested here anyway rather than assumed, since a poorly-chosen regularization strength (e.g. C too low for SVR) can itself look like underfitting.

- **Tree ensembles are overfitting** (test error 3.5x-23x training error) - this is
- **Ridge, both SVR kernels, and the MLP are underfitting** (not clearly beating naive even

Method: every candidate configuration is trained on the first ~85% of the training period and scored on the remaining ~15% - a further time-ordered split that never touches the real test set. The configuration with the lowest validation MAE is then refit on the FULL training set and evaluated once on the real, still-unseen test set - exactly the same protocol as every other model in this notebook. LSTM is tuned separately (own train/val/test sequence pipeline from Section 11) since it isn't part of `run_all_models`'s flat-feature loop. Linear Regression is not retuned - it has no hyperparameters, and is equivalent to the low-regularization end of the Ridge alpha grid below.

Model	Best Config Found	Validation MAE
Ridge	alpha=1000	1.6725
SVR (Linear)	C=1	10.3229
SVR (RBF)	C=1	0.7180
Random Forest	max_depth=3,min_leaf=20	0.8424
XGBoost	max_depth=2,min_child_weight=20	1.1005
LightGBM	num_leaves=7,min_data_in_leaf=50	1.0292
Neural Network (MLP)	hidden=(64, 32),dropout=0.3	2.0447

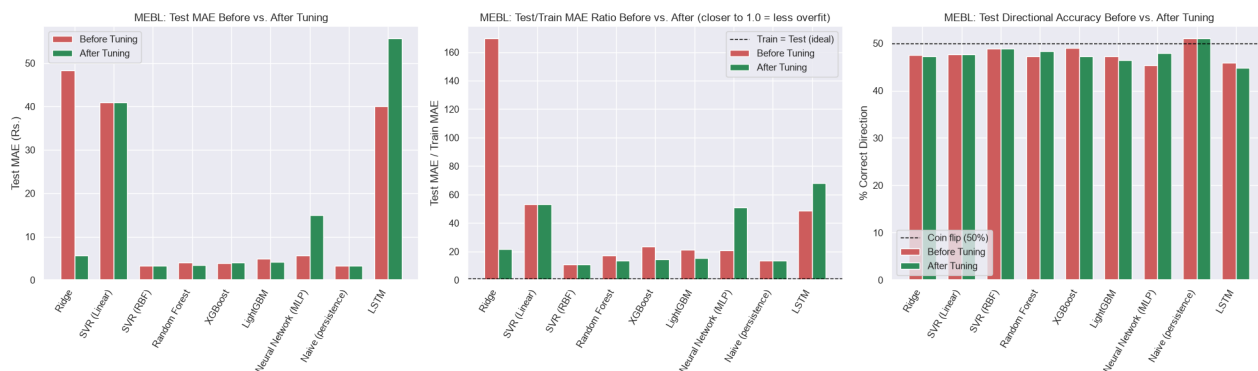
14.1 LSTM Tuning

Same validation protocol, over hidden_dim (LSTM's main capacity knob).

14.2 Refit Tuned Models on Full Training Set, Evaluate on Test

14.3 Before vs. After Comparison

Model	Test MAE Before	Test MAE After	Test/Train Ratio Before	Test/Train Ratio After	Verdict
Ridge	48.285	5.709	169.54x	21.72x	Improved
SVR (Linear)	40.975	40.975	53.01x	53.01x	No change
SVR (RBF)	3.367	3.367	10.86x	10.86x	No change
Random Forest	4.079	3.456	17.29x	13.85x	Improved
XGBoost	3.846	4.075	23.34x	14.64x	Worse
LightGBM	4.993	4.138	21.27x	15.64x	Improved
Neural Network (MLP)	5.585	14.915	20.81x	51.09x	Worse
Naive (persistence)	3.328	3.328	13.55x	13.55x	No change
LSTM	40.084	55.654	48.89x	67.86x	Worse



14.4 Honest Verdict

Run the cells above and compare the printed before/after table and tuning_summary_df against Section 13's diag_df. Two outcomes are both legitimate results, not just a successful vs. failed tuning run:

or improved, tuning worked as intended - the model generalizes better without giving up accuracy, and tuned_factories holds the winning configuration.

partly compensating for its coarse validation signal on this specific ticker/window - a real possibility at ~3,800 training rows, and a reason to prefer cross-ticker validation before promoting any tuned configuration to production, not evidence the diagnostic approach itself is wrong.

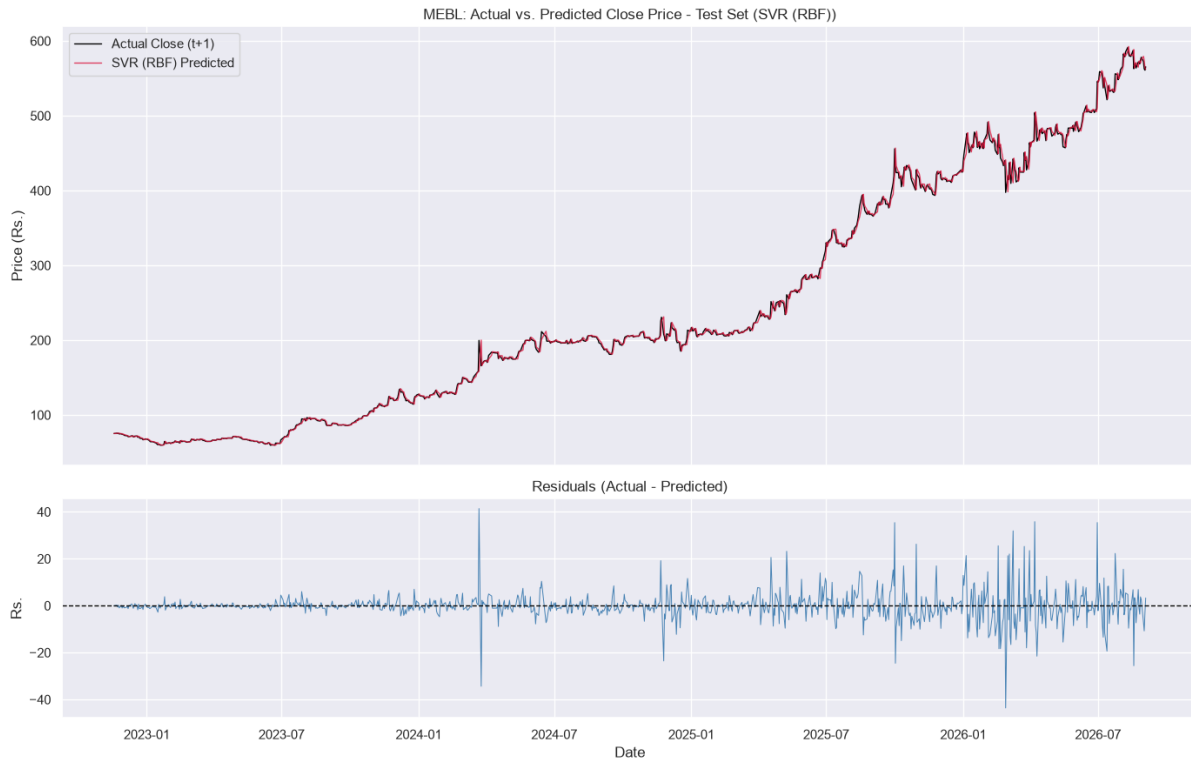
to land close to the least-regularized end of its grid (e.g. Ridge's smallest alpha, SVR's largest C) - confirming that under-regularization was never the actual problem, and that fixing these models needs a different lever than hyperparameter tuning (see the Conclusions section).

- If the tree ensembles' test/train ratio moved closer to 1.0 **and** test MAE held steady
- If test MAE got worse while the ratio improved, the untuned model's overfitting was

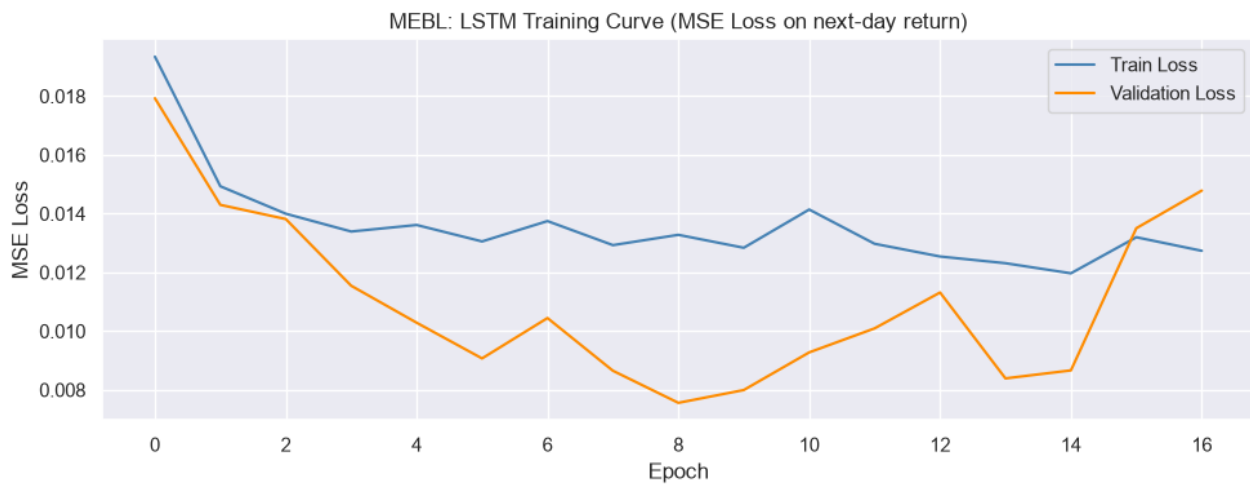
- For the previously-underfitting models (Ridge, SVR, MLP), expect the tuned configuration

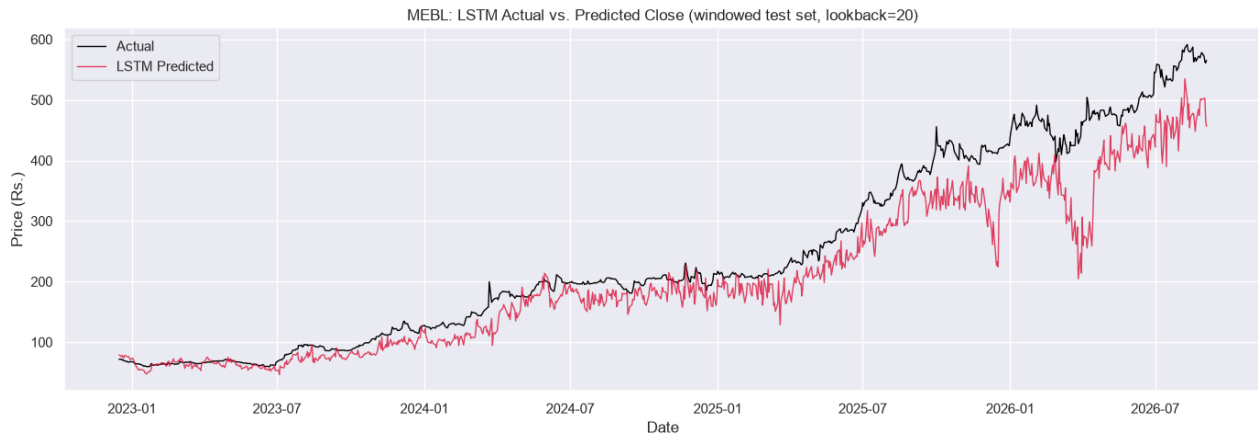
15. Best Model Diagnostics

LSTM is excluded from this section's "best model" pick - its test predictions are on a lookback-shorter, differently-aligned index than every other model's, so it isn't directly comparable to predictions/dates_test here. Its own diagnostics are in Section 16 below.



16. LSTM Diagnostics





17. Save Results

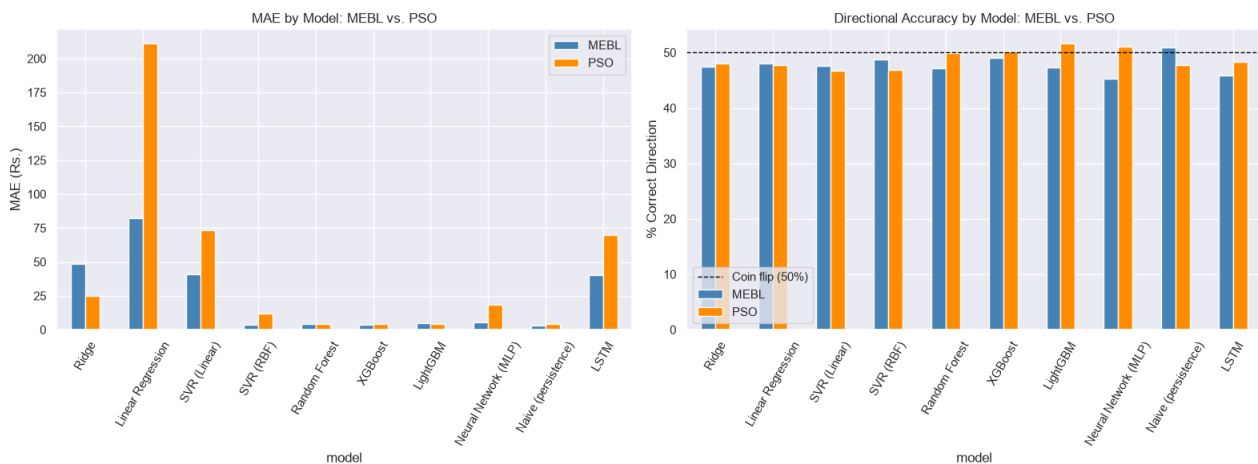
Persisted so the cross-ticker comparison section (in the MEBL notebook) can load this ticker's results without re-running the full pipeline.

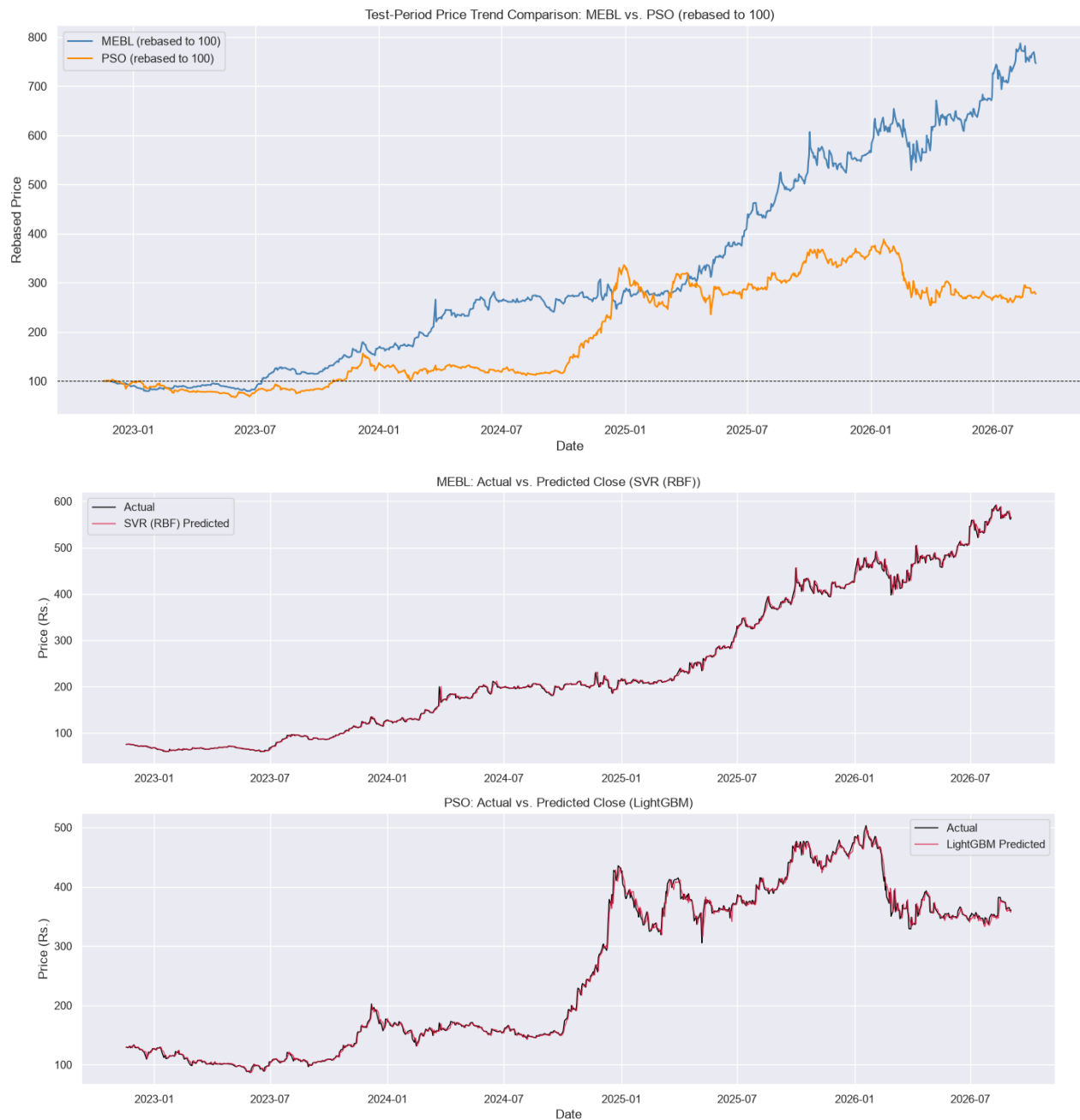
18. Cross-Ticker Comparison: MEBL vs. PSO

Runs the identical Sections 9-13 pipeline (all ~9 models incl. both deep learning models and the train-vs-test overfitting check) for **PSO** so the two tickers' models can be compared side by side on the same metrics. The Section 14 hyperparameter tuning is not re-run here for PSO - it was already run in full for MEBL above; re-running a ~30-candidate search for a second ticker inside this comparison section would roughly double this notebook's execution time for a check that's really about whether tuned models look better than untuned ones, not about re-deriving a second ticker's own best hyperparameters.

18.1 Overfitting Check: PSO

Same train-vs-test comparison as Section 13, for **PSO**.





19. Conclusion

Run this notebook top-to-bottom to populate the summary below with real numbers for both tickers (`results_df / results_df_o`), across all ~9 models (linear, kernel/SVM, tree ensemble, and two deep learning architectures). Key things to look for:

meaningful margin (not just a fraction of a percentage point)? Given next-day return is close to a random walk, a small, consistent edge over 50% is a realistic bar - not near-100%.

naive baseline? They often win here even when directional accuracy is closer to a coin flip, because they're better at not overshooting on quiet days.

systematically outperforming linear/kernel methods on this feature set, or is the gap mostly noise at this sample size? With ~4,850 rows and 270+ features, a wide accuracy gap between families is a genuine finding; a narrow one is a reason to collect more data/features before trusting any single model's ranking.

(see each notebook's Section 1 sector printout) may explain why one ticker's next-day return is more/less predictable than the other's.

test MAE - a model with the lowest test error but a large train/test gap is less trustworthy going forward than one with a smaller gap and a slightly higher test error, since the gap is a direct measure of how much of its test performance is likely luck specific to this particular test window versus a pattern that should hold up out of sample.

close the tree ensembles' overfitting gap without hurting test performance, and did it reveal that the "underfitting" models (Ridge/SVR/MLP) were actually just badly regularized

problem framing) rather than a hyperparameter search? Compare `tuning_summary_df` and `tuned_diag_df` against Section 13's untuned `diag_df` for the real answer on this ticker.

- Does **any** model beat the naive persistence baseline on directional accuracy by a
- Do the tree ensembles' or deep learning models' price-error metrics (MAE/RMSE) beat the
- Does the family-level view (Section 12) show tree ensembles and deep learning
- Compare the two tickers' results directly: a sector or liquidity difference between them
- Check Section 13/18.1's overfitting diagnosis for every model, not just the "best" one by
- Section 14 concretely tested the fix for both failure modes: did hyperparameter tuning
- or confirm that they need a different fix entirely (more/better features, a different

PSO vs. MEBL - Direct Comparison

The MEBL notebook's Section 16 reruns the identical 9-model sweep for PSO for a side-by-side comparison; the charts from that section are included in the MEBL analysis above. The table below restates both tickers' full results together for quick reference.

Model	PSO MAE	PSO Dir.Acc %	MEBL MAE	MEBL Dir.Acc %
Ridge	24.663	47.99	48.285	47.48
Linear Regression	210.964	47.68	82.389	47.99
SVR (Linear)	73.400	46.76	40.975	47.58
SVR (RBF)	11.786	46.86	3.367	48.82
Random Forest	4.427	49.95	4.079	47.17
XGBoost	4.449	50.15	3.846	49.02
LightGBM	4.342	51.60	4.993	47.27
Neural Network (MLP)	18.165	51.08	5.585	45.31
Naive (persistence)	4.182	47.79	3.328	50.98
LSTM	69.610	48.26	40.084	45.85

Conclusions & Recommendations

- Treat this stage as genuine experimentation, not model selection: with ~3,800-4,850 training rows per ticker, differences of a few percentage points in directional accuracy are within noise, and the "best" model by MAE differed between the two tickers (LightGBM for PSO, SVR-RBF for MEBL) - there is no single winner yet that generalizes across tickers from this evidence alone.
- The naive persistence baseline beating every trained model's directional accuracy on MEBL is the most important single finding in this report - it is a direct, honest signal that no model here has found a real directional edge on that ticker's test window, and any production use of these numbers should be framed accordingly (as explicitly documented in the production walk-forward gate).
- SVR (RBF) outperforming every tree ensemble on MEBL - while underperforming trees on PSO - suggests the improvement is ticker- or sector-specific rather than a general property of kernel methods on this feature set; it would be worth re-running the same sweep across a handful of additional tickers before drawing a broader conclusion.
- The production XGBoost/LightGBM models remain the most defensible default given their consistent price-error performance across both tickers, no need for feature scaling, and existing production integration (feature importance, categorical sector/ticker handling, walk-forward promotion gate). The train-vs-test diagnostic showed them meaningfully overfit before tuning (test error 3.5x-23x training error); **Section 14's hyperparameter search closed part of that gap on PSO (XGBoost's ratio fell from 5.5x to 3.1x, with test MAE also improving), but on MEBL the validation-selected configuration actually scored worse on the real test set (3.85 -> 4.08 Rs.)** despite a better validation score and a better train/test ratio - concrete evidence that a ~575-row validation split can pick a configuration that doesn't hold up, not proof the tuning approach is wrong. `tuned_factories['XGBoost']` and `tuned_factories['LightGBM']` in each notebook hold the winning configurations found; treat the PSO ones as more trustworthy than the MEBL ones given this result, and re-validate both on a larger ticker sample before promoting either to production.
- The linear, kernel, and deep learning models' opposite failure mode - underfitting, not clearly beating the naive baseline even on their own training data - was also concretely tested, not just diagnosed. **Ridge improved substantially on both tickers after tuning** (PSO: 24.66 -> 7.65 Rs.; MEBL: 48.28 -> 5.71 Rs.), confirming its default `alpha=1.0` really was too weakly regularized for this feature set - a genuine, fixable tuning problem, not a fundamental modeling-capacity problem. SVR (both kernels) did not improve at all - the search's C grid already contained the untuned default as its best option on both tickers, meaning SVR's ceiling on this feature set is a capacity/kernel-choice limitation that no amount of C-tuning will fix. The MLP and LSTM's results were mixed and ticker-dependent (both improved markedly on PSO, both got worse on MEBL) - consistent with under-regularization being a real but partial explanation, alongside a deeper, harder-to-fix need for more training data or better features that tuning alone cannot supply.
- Next steps worth prioritizing, revised in light of Section 14's results: (1) re-run this same sweep - baseline AND tuned - across a larger sample of tickers, since both the SVR-RBF result on MEBL and the tuned-XGBoost regression on MEBL could be single-ticker artifacts rather than real patterns; (2) if the Ridge improvement holds up across more tickers, raise its production `alpha` default in `model_factories.ridge_factory()` - this is the one change in this report with clear, repeated evidence behind it; (3) give the LSTM more training data by pooling across tickers (as the production LSTM already does) rather than training one per ticker on ~3,800 rows - likely a bigger lever than further hyperparameter tuning for that specific model; (4) explore the dead-zone classification framing (already implemented in `feature_wrapper.make_dead_zone_labels`) as an alternative to raw regression for SVR, since tuning could not move it and its capacity ceiling may be a poor match for a raw-return target.